

ATmega32A Module

ADC (Analog-to-Digital Converter)

Ali Sahafi

1 Theory

The ADC converts a voltage into a number. The ATmega32A has one **10-bit** ADC with an 8-channel multiplexer on **PORTA** (ADC0–ADC7 = PA0–PA7): one converter, eight selectable inputs.

With 10 bits the result is 0–1023, measured relative to a **reference voltage** V_{ref} :

$$\text{result} = \frac{V_{\text{in}}}{V_{\text{ref}}} \cdot 1024 \quad \Longleftrightarrow \quad V_{\text{in}} = \frac{\text{result}}{1024} \cdot V_{\text{ref}}$$

Three reference options: the **AREF** pin (ADC_REF_AREF), the supply voltage **AVCC** (ADC_REF_AVCC, the usual choice) or the internal 2.56 V bandgap (ADC_REF_INTERNAL).

Prescaler. The ADC needs a clock between 50 and 200 kHz for full resolution. The prescaler divides F_{CPU} down into that window: at 8 MHz, prescaler 64 gives 125 kHz — the driver default. One conversion takes about 13 ADC clock cycles ($\approx 100 \mu\text{s}$).

2 Registers

Register	Role	Bits used by the driver
ADMUX	Reference + channel select	REFS1:0, MUX2:0
ADCSRA	Control and status	ADEN, ADSC, ADATE, ADIE, ADPS2:0
ADCL/ADCH	10-bit result	read ADCL first, then ADCH!
SFIOB	Auto-trigger source	ADTS2:0 = 000 (free running)

3 API reference

Method	Description
ADC.init(ref, prescaler, interrupt)	Configure reference, prescaler, optional interrupt
ADC.read(channel)	Blocking read — waits for conversion, returns 0–1023
ADC.startConversion(channel)	Non-blocking single-shot conversion
ADC.startContinuous(channel)	Free-running mode — auto-converts continuously
ADC.stopContinuous()	Stop free-running mode
ADC.isReady()	Returns true when conversion is complete
ADC.lastResult()	Read last result without triggering a new conversion
ADC.stop() / ADC.start()	Disable / re-enable the ADC
ADC.attachInterrupt(callback)	Set conversion callback: <code>void cb(uint16_t value)</code>

- **Reference:** ADC_REF_AREF, ADC_REF_AVCC, ADC_REF_INTERNAL
- **Prescaler:** ADC_PRESCALER_2/4/8/16/32/64/128
- **Channel:** ADC_CHANNEL_0 ... ADC_CHANNEL_7

Three usage styles, from simplest to most advanced:

- **blocking:** `read()` — the CPU waits for the conversion
- **polling:** `startConversion()`, then `isReady()` + `lastResult()`
- **interrupt-driven:** `startContinuous()` + callback — see the capstone `example.cpp` in the repository root

4 Example: potentiometer bar graph

Wiring.

- Potentiometer ($\geq 1\text{k}\Omega$): outer pins to VCC and GND, wiper to **PA0**.
- 8 LEDs + $330\ \Omega$ resistors from **PC0–PC7** to GND.
- **AVCC must be connected to VCC** — the ADC is powered from the AVCC pin.

Build & flash. `make adc`

```
/*
 * ADC Example -- potentiometer bar graph
 *
 * Reads a potentiometer on ADC0 (PA0) with a blocking read and shows the
 * value as a bar of 0-8 LEDs on PORTC.
 *
 * Wiring:
 * - Potentiometer: outer pins to VCC and GND, wiper to PA0
 * - 8 LEDs + 330R resistors from PC0..PC7 to GND
 * - AVCC connected to VCC (the ADC is powered from the AVCC pin!)
 *
 * Build & flash: make adc
 */

#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "adc.hpp"

int main() {
    GPIO.setDirection(DDRA, PA0, INPUT); // ADC0 input
    GPIO.setDirection(DDRC, ALL, OUTPUT); // LED bar

    ADC.init(ADC_REF_AVCC, ADC_PRESCALER_64);

    while (true) {
        uint16_t value = ADC.read(ADC_CHANNEL_0); // 0..1023

        // Map 0..1023 to 0..255 for 8 LEDs
        uint8_t bar = (value * 255) / 1023;
        GPIO.write(PORTC, bar);

        _delay_ms(50);
    }

    return 0;
}
```

5 Exercises

1. The pot is at mid-position. What value does `ADC.read()` return (roughly), and what voltage does that correspond to at $V_{\text{ref}} = 5\text{ V}$?
2. Rewrite the example in the *polling* style (`startConversion` / `isReady` / `lastResult`) so the LED bar updates while the main loop stays free to blink a ninth “heartbeat” LED.
3. Connect an LM35 temperature sensor ($10\text{ mV}/^{\circ}\text{C}$) to ADC1 and compute the temperature in degrees. Which reference gives better resolution for this sensor: AVCC (5 V) or internal 2.56 V?